

OceanClaw

?? ?? AI Agent ?????

2026? ??????? X ICT ??? ?? ???? ????? ??

PDF RAG ? Ollama Local LLM ? FAISS ? FastAPI ? Obsidian Wiki ? Jetson Orin Nano

?? ??: ?? OceanClaw ?? ??? ? Jetson ?? ?? ??

1. 프로젝트 개요

항목	내용
프로젝트명	OceanClaw - 선박 정비 AI Agent
개발 목적	선박 정비 매뉴얼, 센서 데이터, 정비 지식을 통합 검색하여 정비 담당자가 빠르게 근거 기반 답변을 얻을 수 있는 로컬 AI Agent 구현
핵심 기술	PDF RAG, FAISS Vector Search, Ollama Local LLM, FastAPI, Obsidian Wiki, NVIDIA Jetson Orin Nano Super Developer Kit
실행 환경	Windows 개발 PC, NVIDIA Jetson Orin Nano Super Developer Kit, Python, Ollama, FAISS
주요 산출물	PDF/CSV 검색 파이프라인, 로컬 LLM 답변 생성, FastAPI 서버, Obsidian 호환 Wiki, Jetson 배포 환경

OceanClaw는 선박 정비 현장에서 필요한 기술 매뉴얼, 센서 이벤트, 정비 지식 문서를 검색하고 답변하는 Offline-First AI Agent 프로젝트이다. 선박 환경은 인터넷 연결이 불안정할 수 있으므로 외부 API 의존도를 낮추고, Ollama 기반 로컬 LLM과 FAISS 검색 엔진을 사용해 Jetson Orin Nano에서도 실행 가능한 구조를 목표로 하였다.

2. 개발 배경 및 필요성

선박 정비 업무에서는 장비 매뉴얼, 점검 절차, 과거 정비 이력, 센서 이상 이벤트를 함께 확인해야 하는 경우가 많다. 그러나 실제 현장에서는 PDF 매뉴얼이 길고, 필요한 페이지를 빠르게 찾기 어렵고, 센서 데이터와 매뉴얼 근거를 함께 해석하는 데 시간이 많이 소요된다.

본 프로젝트는 이러한 문제를 해결하기 위해 다음 기능을 제공한다.

- ? 선박 장비 매뉴얼 PDF에서 관련 페이지와 절차 검색
- ? 센서 CSV 데이터를 점검 이벤트 형태로 변환하고 검색
- ? 검색 결과를 기반으로 한국어 정비 답변 생성
- ? 모든 답변에 출처와 검색 근거 표시
- ? Obsidian 호환 Markdown Wiki로 정비 지식 축적
- ? Jetson Orin Nano에서 로컬 서버로 실행하여 외부 PC에서 접속

3. 개발 목표

3.1 기능 목표

구분	목표
PDF 처리	선박 매뉴얼 PDF를 페이지 단위로 추출하고 chunk로 분할
검색	Ollama embedding과 FAISS를 이용한 dense retrieval 구현
답변 생성	검색된 근거만 사용하여 한국어 답변 생성
센서 데이터 처리	원본 CSV를 OceanClaw용 sensor event CSV로 변환

구분	목표
Wiki	Obsidian에서 열 수 있는 Markdown 지식 문서 생성 및 검색
API	FastAPI 기반 /health, /ask, /search/manual, /search/sensor 제공
배포	Jetson Orin Nano에서 API 서버 실행 및 동일 네트워크 외부 접속 확인

3.2 비기능 목표

- ? 인터넷 연결이 불안정한 환경을 고려한 로컬 실행 구조
- ? Gemini 등 유료 API 의존도를 낮춘 Ollama 기반 개발
- ? 데이터 처리 결과를 JSONL/CSV/FAISS로 분리하여 검증 가능하게 구성
- ? Jetson의 제한된 연산 성능을 고려하여 top-k, timeout, route 옵션 제공

4. 시스템 구성

- ■■
 - > Query Expansion
 - > Manual FAISS Search
 - > Sensor FAISS Search
 - > Wiki FAISS Search
 - > Context ■■
 - > Ollama LLM ■■ ■■
 - > ■■■ ■■ ■■ ■■

4.1 주요 구성 요소

구성 요소	설명
pdf_extract.py	PDF를 페이지별 JSONL로 추출
chunk.py	LangChain Text Splitter 기반 chunk 생성
vectorstore.py	FAISS index 생성 및 검색
query_expansion.py	한국어 정비 질문을 영어 검색어로 확장
answer.py	검색 결과 기반 답변 생성
api.py	FastAPI 서버
wiki_writer.py	RAG 근거 기반 Markdown Wiki 생성
wiki_index.py	Obsidian Wiki Markdown 검색 index 생성

5. 데이터 처리 및 RAG 구현

5.1 PDF 매뉴얼 처리

Yanmar 6LF Series Operation Manual PDF를 pypdf로 페이지별 추출하여 data/interim/pdf_pages.jsonl로 저장하였다. 이후 LangChain RecursiveCharacterTextSplitter를 사용하여 chunk를 생성하였다.

주요 산출물은 다음과 같다.

```
data/interim/pdf_pages.jsonl
data/processed/pdf_chunks.jsonl
index/pdf.faiss
index/pdf_docs.json
```

PDF 검색 테스트 결과, "엔진 오일 점검 방법" 질문에 대해 `yanmar_6lf_operation_manual.pdf` p.69의 `Check Oil Level in Engine` 항목이 검색되었다.

5.2 센서 데이터 처리

원본 `data.csv`를 OceanClaw용 `sensor_events.csv`로 변환하였다. 원본 데이터는 정비 로그 형식이 아니었기 때문에, compressor decay와 turbine decay 값을 기준으로 watch/warning 이벤트를 생성하였다.

변환 기준 예시는 다음과 같다.

기준	이벤트
compressor decay 감소	GT Compressor watch/warning
turbine decay 감소	GT Turbine watch
running hours	이벤트 발생 시점 기록

주요 산출물은 다음과 같다.

```
data/processed/sensor_events.csv
index/sensor.faiss
index/sensor_docs.json
```

5.3 Query Expansion

매뉴얼이 영어이기 때문에 한국어 질문만으로는 검색 정확도가 낮았다. 이를 보완하기 위해 정비 용어 기반 query expansion을 구현하였다.

예시:

```
■■ ■■ ■■ ■■
-> ■■ ■■ ■■ ■■ engine oil level dipstick MIN MAX check inspect inspection procedure how to
```

이를 통해 한국어 질문에서도 영어 매뉴얼의 관련 페이지를 안정적으로 검색할 수 있었다.

6. LLM 답변 생성

답변 생성은 Ollama chat API를 사용하였다. 초기 계획서에는 Google Gemini 사용이 포함되어 있었으나, 비용과 인터넷 의존성을 줄이고 Jetson 로컬 실행을 고려하여 Ollama 기반으로 변경하였다.

현재 기본 모델은 다음과 같다.

용도	모델
Embedding	nomic-embed-text
Chat	gemma3:4b

답변 생성 원칙은 다음과 같다.

- ? 검색된 context 안의 정보만 사용
- ? 절차는 번호 목록으로 정리
- ? 센서 이벤트는 component, severity, running hours, symptom, recommended action 포함
- ? 출처는 프로그램이 검증된 source/page/event_id를 붙임

7. FastAPI 서버 구현

Jetson 외부 접속과 팀원 테스트를 위해 FastAPI 서버를 구현하였다.

제공 API는 다음과 같다.

Endpoint	설명
GET /health	서버와 Ollama 연결 상태 확인
POST /ask	질문에 대한 RAG 답변 생성
POST /search/manual	PDF 매뉴얼 검색
POST /search/sensor	센서 이벤트 검색

실행 명령:

```
python scripts/run_api.py --host 0.0.0.0 --port 8000
```

데스크탑에서는 다음 주소로 접속한다.

```
http://<Jetson-IP>:8000/docs
```

8. Obsidian LLM Wiki 구현

계획서의 LLM Wiki 및 Obsidian 요구사항을 반영하기 위해 wiki/ 폴더를 Obsidian Vault처럼 사용할 수 있도록 구성하였다.

```
wiki/  
  components/  
  procedures/  
  logs/  
  templates/
```

예시 문서:

```
wiki/components/engine_oil.md  
wiki/components/dipstick.md  
wiki/procedures/check_engine_oil_level.md  
wiki/logs/gt_compressor_warning.md
```

Wiki 문서는 Obsidian 내부 링크를 사용한다.

- [[engine_oil]]
- [[dipstick]]
- [[check_engine_oil_level]]

또한 generate_wiki.py를 통해 RAG 검색 근거 기반 Markdown 문서를 자동 생성할 수 있다.

```
python scripts/generate_wiki.py "■■ ■■ ■■" --route manual --type procedure
```

생성된 Wiki 문서도 FAISS index로 변환하여 검색 가능하게 구성하였다.

```
data/processed/wiki_chunks.jsonl
index/wiki.faiss
index/wiki_docs.json
```

9. Jetson Orin Nano 배포 및 외부 접속

NVIDIA Jetson Orin Nano Super Developer Kit에 Ubuntu/JetPack 환경을 구성하고 OceanClaw를 배포하였다. Jetson에서 OceanClaw API 서버를 0.0.0.0:8000으로 실행하여 같은 네트워크의 데스크탑에서 접속할 수 있도록 하였다.

확인된 사항:

- ? Jetson 부팅 및 SSH 접속 성공
- ? OceanClaw GitHub 코드 pull 성공
- ? Python 가상환경 구성
- ? Ollama 실행 및 모델 호출 확인
- ? FastAPI 서버 실행
- ? 데스크탑에서 `http://<Jetson-IP>:8000/docs` 접속 성공
- ? /ask API를 통한 매뉴얼 기반 답변 생성 성공

테스트 요청 예시:

```
{
  "question": "■■ ■■ ■■ ■■ ■■■",
  "top_k": 1,
  "min_score": 0,
  "route": "manual"
}
```

응답 결과는 매뉴얼 p.69를 출처로 포함하며, 엔진 오일 점검 절차를 한국어로 반환하였다.

10. 문제 해결 및 개선 사항

10.1 Jetson 설치 문제

초기 JetPack 설치 과정에서 DP to HDMI 변환, USB 불안정, 설치 중 검은 화면 문제가 발생하였다. 이를 해결하기 위해 다음을 수행하였다.

- ? 64GB USB 3.x 장치로 설치 USB 재생성
- ? NVMe SSD 장착 후 설치 대상 확인
- ? 설치 중 GitHub SSH key import와 불필요한 네트워크 옵션 생략
- ? 설치 완료 후 SSH 기반으로 작업 전환

10.2 Jetson timeout 문제

Jetson에서는 데스크탑보다 LLM 응답이 느려 Python API 호출에서 timeout이 발생할 수 있었다. 이를 해결하기 위해 기본 OLLAMA_TIMEOUT을 60초에서 300초로 늘렸다.

```
OLLAMA_TIMEOUT = int(os.getenv("OLLAMA_TIMEOUT", "300"))
```

또한 --route manual을 지정해도 manual과 sensor 검색을 모두 수행하던 문제를 수정하여, route 지정 시 필요한 검색만 수행하도록 개선하였다.

10.3 GitHub 배포 구조 변경

Jetson에서 git pull만으로 실행에 가까운 환경을 만들기 위해, 민감하지 않은 실행 산출물인 data/와 index/를 GitHub에 포함하도록 .gitignore를 조정하였다.

계속 제외하는 항목:

```
.env  
.venv/  
__pycache__/  
*.pyc  
artifacts/
```

11. 현재 구현 상태

기능	상태
PDF 추출	완료
LangChain 기반 chunk	완료
PDF FAISS 검색	완료
센서 CSV 변환	완료
센서 FAISS 검색	완료
Ollama 답변 생성	완료
FastAPI 서버	완료
Jetson 배포	1차 완료
Obsidian Wiki 구조	완료
Wiki 자동 생성	완료
Wiki 검색 index	완료
manual/sensor/wiki 통합 답변	진행 예정
Mattermost 연동	진행 예정

12. 향후 개발 계획

1. manual + sensor + wiki 통합 답변 연결
2. Mattermost 채널 연동
3. Jetson 서버 자동 실행 설정
4. 공유기 DHCP 예약 또는 static IP 설정
5. 응답 시간 및 성능 평가 자동화

6. 실제 선박 정비 데이터 확보 후 평가 질문 세트 작성
7. 기술 용어 번역 개선

13. 기대 효과

OceanClaw는 선박 정비 담당자가 긴 매뉴얼을 직접 탐색하지 않고도 필요한 점검 절차와 근거 페이지를 빠르게 확인할 수 있도록 지원한다. 또한 센서 이벤트와 Wiki 지식을 함께 활용함으로써 반복되는 이상 징후와 관련 정비 절차를 연결할 수 있다.

특히 Jetson 기반 로컬 배포를 통해 인터넷 연결이 제한적인 선박 환경에서도 AI 기반 정비 지원 시스템을 운용할 수 있는 가능성을 확인하였다.

14. 결론

본 프로젝트는 선박 정비 매뉴얼 PDF, 센서 이벤트 CSV, Obsidian Wiki를 통합하는 로컬 RAG 기반 AI Agent를 구현하였다. 개발 과정에서 Gemini 중심의 초기 계획을 비용과 오프라인 실행 조건에 맞추어 Ollama 기반 구조로 전환하였고, Jetson Orin Nano에서 실제 API 서버를 실행하여 외부 PC에서 접속 가능한 상태를 확인하였다.

현재 OceanClaw는 매뉴얼 검색과 답변 생성, 센서 이벤트 검색, Wiki 생성 및 검색, Jetson 배포까지 완료되었으며, 향후 Mattermost 연동과 통합 Agent 구조를 추가하여 계획서의 최종 목표에 더 가까운 현장형 정비 AI Agent로 확장할 예정이다.